FalseGreen

VERIFICATION · SIGNED EVIDENCE

FalseGreen Verification Report

ACCEPTED

All required criteria passed for the exact source tree built and verified by FalseGreen.

This does not prove universal correctness. Assurance is limited to the explicit formal properties, assumptions, models, summaries, and scope recorded in this report.

THE WORK

Job	`951090fd-bba3-4a91-bb78-45f5c49ad0c3`
Replaces (supersede)	`3bd7af61-3786-4405-a28c-76419d3360ee` — terminal verdict: accepted
Language	rust
Toolchain	rustc 1.99.1 (rustc) (1.99.1)
Verifier environment	rust - rustc 1.99.1 (rustc) (1.99.1)

THE FROZEN BOUNDARY

Source tree	`bb48e33ef868c4155edde9e1b4eb0806dd783a22580a6b46f5bab78a5a25f3c0`
Contract SHA-256	`1e230d8f4b97491eafe1e950dcab732695a552925ab398123e495bcc97c6b731`
Criteria SHA-256	`8158e6e90cc6140339c6de235576f60cf79a1890e255df224c199d1ae825f192`
Criteria	4 / 4 PASSED

WHAT FALSEGREEN DID

Attempts	1
Duration	1m 34s
Report schema	v13
Report record	1 (the verdict mint)
Signed	2026-10-26 07:47:53 UTC

Contract (frozen)

Wall budget	2700s

Criterion	Title	Required	Result	Duration
`build`	the workspace compiles (release)	yes	**PASS**	21.3s
`tests`	the full test suite passes offline	yes	**PASS**	15.9s
`clippy`	the workspace is clippy-clean (-D warnings)	yes	**PASS**	7.6s
`formatting`	the tree is rustfmt-clean	no	**PASS**	0.2s

Invariants	chain linking is exact successor arithmetic and never wraps at the u64 ceiling — `ladder.l1-kani, ladder.l2-creusot`

Plan (frozen)

Objective	The 0.15.0 launch artifact for the rust-demo ledger workspace, verified through the proof ladder on the rebuilt job image: the mechanical floor (relea se build, the FULL test suite, clippy -D warnings, rustfmt) as contract criteria, and exactly one claim about the program — 'chain linking is exact successor arithmetic and never wraps at the u64 ceiling' — carried by two formal encodings the machines discharge: Kani L1 (bounded model check over the SHIPPED arithmetic in crates/ledger-rules, full u64 domain) and Creusot L2 (u nbounded Why3 proof of the guarded Pearlite spec inside crates/ledger-spec). 0.15.0 is the first release in which BOTH ladder rows must carry the machine's claim VERBATIM (the clauses extracted from the frozen encoding source, committed under a real encoding_sha256) plus the per-obligation prover disclosure; an empty-string commitment is a defect, not a pass. The 0.11.0-era claims that 'proved' themselves with a mechanical check are deliberately NOT carried: a passing test proves the test passed.
Plan digest	`dc671386fa3d8a1f6e6f0b27fb8d9a337e88f4bc3f0ba018ab7830113ad3c44d`
Work items	4

Work items, in dependency order:

Work item	Outcome	Depends on	Omission risk
`criteria-floor`	The mechanical floor runs as criteria on every attempt: the workspace compiles in release, the full test suite passes, the workspace is clippy-clean at -D warnings, and the tree is rustfmt-clean (optional).	-	a criterion that matches nothing would pass vacuously — each argv is the whole suite or a build, not a test-name filter
`ladder.l1-kani`	Kani L1 decides the claim over the SHIPPED arithmetic (proofs/links_kani.rs pulls crates/ledger-rules/src/lib.rs in by path, so the bytes the binary runs are the bytes proved): the successor predicate never wraps at the u64 ceiling, and on the guarded domain it is exactly seq == prev + 1.	criteria-floor	a harness that copies the code instead of including it would prove a copy, not the shipped arithmetic; a harness that explodes in std machinery would come back UNAVAILABLE and hold the Job at insufficient_evidence
`ladder.l2-creusot`	Creusot L2 proves the guarded spec (#[requires(prev < u64::MAX)] + #[ensures(result == (seq == prev + 1u64))] inside crates/ledger-spec) for ALL inputs — unbounded, so it covers what the bounded check cannot.	criteria-floor	an empty or non-matching target list yields zero goals — the rung must be recorded, never silently passed; the workspace-root src/ scan does not see a member crate's spec (0.14.0 finding #33), so the claim TEXT may be missing from the row even though the proof closed; a session-path target (`verif/<crate>_rlib/<item>`) used to blind the clause extractor (the whole path was taken as the item name), yielding a proven row with NO claim text and an empty-string digest — pinned by a unit test in 0.15.0 and by this row
`report-artifact`	The Job terminates `accepted` with both rung rows and all five security classes satisfied; the signed JSON + the human rendering become the marketing artifact (saved in the repo, locally verified keyless).	ladder.l1-kani, ladder.l2-creusot	a security-class shortfall (secret/SCA/SC) would verdict the Job insufficient_evidence or failed regardless of the proofs

Acceptance conditions, and the evidence that decides each:

Scope	Condition	Evidence
`criteria-floor`	the release build of the whole workspace succeeds inside the job container	build
`criteria-floor`	the full workspace test suite passes in the job container (the shipped ledger behaviour, unfiltered)	behavioral-test
`criteria-floor`	cargo clippy --workspace -- -D warnings exits 0 inside the job container (the lint bar the marketing artifact claims)	build
`ladder.l1-kani`	Kani L1 decides the claim and the signed row states HOW: decision `proven`, the pinned tool version from the image manifest (kani 0.68.0), the harness clauses extracted verbatim, and a real encoding_sha256 over the frozen harness bytes.	proof
`ladder.l2-creusot`	Creusot L2 discharges the guarded spec and the signed row states HOW: decision `proven`, the pinned version (creusot v0.14.0 + the prover set), the WHY3 obligation closed, the Pearlite clauses extracted VERBATIM from the member crate (crates/ledger-spec/src/lib.rs — the 0.15.0 extraction fix), a real encoding_sha256, and the per-obligation prover disclosure (prover + time + status).	proof
`report-artifact`	the terminal verdict is `accepted`, the report carries both invariant rows with decisions, and `falsegreen_verify_report` verifies the saved JSON locally (Ed25519, keyless)	artifact
`overall`	one Job, one entitlement, terminal `accepted`: the mechanical criteria green, the Kani rung proven (bounded), the Creusot rung proven (unbounded), all five mandatory security classes satisfied, and the saved signed report verifies locally.	proof

Evidence manifest

ATTEMPT 1 - PASSED - 2026-10-26 07:47:53 UTC - WALL 1M 34S

source sha256: bb48e33ef868c4155edde9e1b4eb0806dd783a22580a6b46f5bab78a5a25f3c0

env sha256: a4a5bf077e04f6aeb04a2ea1c0b0852414491d85daa6909080ad17cbf5f7f2e8

Security Findings

The bar these findings are judged against (the frozen policy): **fail on:** critical · **unknown severity:** insufficient_evidence.

Nothing to act on — 180 recorded facts (are inventory evidence in the signed JSON report).

The full SBOM and VEX are in the signed JSON report.

JSON report: https://falsegreen.com/report/951090fd-bba3-4a91-bb78-45f5c49ad0c3/json

Invariant Checks

Each entry is ONE frozen claim, the PROVER that worked on it, and the formal proposition that prover was asked to discharge — read out of the frozen encoding, so the machine's text sits next to the author's sentence. The full rows (any counterexample trace, the admitted assumptions) are in the signed JSON report.

ATTEMPT #1

chain linking is exact successor arithmetic and never wraps at the u64 ceiling

PROVEN

"record N links to record N-1 iff prev_hash equals record N-1's hash AND seq is the exact u64 successor of record N-1's seq; the successor is never taken modulo the u64 ceiling — no seq may link past u64::MAX (the chain cannot wrap) — and on the guarded domain (prev below u64::MAX) the predicate is exactly seq == prev + 1"

user-provided description — not machine-checked

Formal claim:

```
kani::assume(seq < u64::MAX)
kani::assert(!wrapped)
kani::assume(prev < u64::MAX)
kani::assert(got == (seq == prev.wrapping_add(1)))
```

Harness: `successor_never_wraps_at_the_u64_ceiling`

`encoding sha256: e87bb002ddad5cf216c2e7df97470f826a80205820eff0e66ffd16bcf5c19d24`

decided by: kani 0.68.0 (bounded)

chain linking is exact successor arithmetic and never wraps at the u64 ceiling

PROVEN 1 / 1

"record N links to record N-1 iff prev_hash equals record N-1's hash AND seq is the exact u64 successor of record N-1's seq; the successor is never taken modulo the u64 ceiling — no seq may link past u64::MAX (the chain cannot wrap) — and on the guarded domain (prev below u64::MAX) the predicate is exactly seq == prev + 1"

user-provided description — not machine-checked

Formal claim:

```
#[ensures(result == (seq == prev + 1u64))]
```

```
#[requires(prev < 18446744073709551615u64)]
```

Target: `verif/ledger_spec_rlib/links`

`encoding sha256: 92ba29b838925b5024384e389e748475d9bb4e41d1df99beb791ab953efcad62`

provers: alt-ergo closed Coma::vc_links (11 ms)

decided by: creusot v0.14.0 (65349025c67e) (unbounded) on alt-ergo

Security

Policy	artifact-0150
Policy digest	`94a8ea9e583126918d71000d5df6184ed14e48f81e1aba1f0aefbc26cb3bb5e3`
Result	SATISFIED

Evidence class	Required	Status	Reasons
Dependency/SCA	yes	**PASS**	
Static analysis	yes	**PASS**	
Secrets	yes	**PASS**	
Supply chain	yes	**PASS**	92 components inventoried (evidence, in the signed JSON report)
License policy	yes	**NOT ASSESSED**	no licence policy frozen — the licence facts are inventory, nothing was asserted · 88 licence facts inventoried (evidence, in the signed JSON report)

How to read this table. **PASS** — the evidence was collected and nothing in it violated the frozen policy. **FAIL** — the evidence showed a violation the policy forbids; that alone fails the Job. **INSUFFICIENT EVIDENCE** — the evidence could not be collected, or could not be decided for a reason outside the code (a stale snapshot, an unavailable tool, output the parser could not read). It is never a pass, and it holds the Job at *insufficient_evidence* rather than accepting it. **NOT ASSESSED** — no policy is frozen for this class, so nothing was asserted about it; the facts are recorded as inventory and the class does not hold the Job. The findings table lists only what the frozen policy JUDGED — a graded severity, supply-chain drift, or a licence breaking the frozen policy. Everything else the tools recorded (components, licence facts) is inventory: not a finding, counted on the class row above, kept in the signed JSON report. A finding is **OPEN** unless it is resolved, not affected, or covered by a recorded exception.

Attempt security history (the full per-attempt sections are in the signed JSON report):

Attempt	Result	Open findings	Demonstrated violations
#1	**SATISFIED**	0	—

Witnesses (tool provenance)

```
osv-scanner 2.7.0 (Dependency/SCA) — invocation sha256:
47f71b51c6d6860e40394682bf283aedfb2c6837ea30a7af48ec466577ff9e63 —
a4a5bf077e04f6aeb04a2ea1c0b0852414491d85daa6909080ad17cbf5f7f2e8
```

semgrep 1.181.0 (Static analysis) — invocation sha256: d044c64e612294961f9688a4216174055f8368dc71fb5f8df1b4ba2b21824b0c — a4a5bf077e04f6aeb04a2ea1c0b0852414491d85daa6909080ad17cbf5f7f2e8

gitleaks 8.30.1 (Secrets) — invocation sha256: 38a4bff4810a10fd9fdde3b3189578097a42ecf4ad1d5cebc4a2f5f54ce638c6 — a4a5bf077e04f6aeb04a2ea1c0b0852414491d85daa6909080ad17cbf5f7f2e8

syft 1.54.1 (Supply chain) — invocation sha256: 29aedc7a59da7a03e51542c7949ea9d1d469979c1af5fe989d6da04fda117a8d — a4a5bf077e04f6aeb04a2ea1c0b0852414491d85daa6909080ad17cbf5f7f2e8

dependency-manifest 1 (License policy) — invocation sha256: 6608f15bd8e2aadb54601ef1ab92b808fd7ae0a72cfa98e481bed31c1076bfed — a4a5bf077e04f6aeb04a2ea1c0b0852414491d85daa6909080ad17cbf5f7f2e8

Tools & provers used

Tool	Version	What it did in this run
osv-scanner	`2.7.0`	Dependency/SCA evidence witness
semgrep	`1.181.0`	Static analysis evidence witness
gitleaks	`8.30.1`	Secrets evidence witness
syft	`1.54.1`	Supply chain evidence witness
dependency-manifest	`1`	License policy evidence witness
kani	`0.68.0`	invariant proofs (1 row)
creusot	`v0.14.0 (65349025c67e)`	invariant proofs (1 row)
alt-ergo, z3	`not recorded (image-pinned)`	authorized provers (frozen contract)

Exports

Full content is embedded in the signed JSON report (report schema v13).

spdx-3.0 sha256:c4802d0199cf9ea30f26db5f60a0798ad97089a6f74c9fe8f72bdb0563f2cd09 embedded in the JSON report (66947 bytes)

cyclonedx-1.7 sha256:c564891997d168c259a8a3539623504c2000354160dc71784d567930569270da embedded in the JSON report (32209 bytes)

cyclonedx-1.7-vex sha256:6a5ed5d51771a20b76021330a5ada80f2dd0661a499954848fdf2b66ca25f33e embedded in the JSON report (632 bytes)

Signature

This report is cryptographically signed: Ed25519 over the canonical report content. The public key is embedded in the report — verification is offline and registry-free (`falsegreen verify-report`).

```
key id: fg-report-9fc1a2c5-134c-4ca1-a0f2-56b2d1bcde67-20261018-1
ed25519 public key: ie4eQdY1lt7O5aCCQzO+/192Yzc4pLQsGyVNqSKtlmA=
ed25519 signature: 8/mfBUgDoE7lgIr/LRtFWserI34hcfr4G0MijxDK5tAtkUx9eAmMnjAWezA85IMH1RRKs8v3KpAb17HClq06AA==
digest: 53369fab29059e1f1ee461124c4bc6817a4ed74835bfcf7537ea038b18efd04e
```

This is a presentation of the signed canonical evidence (the JSON artifact). It is not an audit certificate. - FalseGreen v0.15.0